Software Engineering Project

DispatchNet:

Alessio Zanon
Leonardo Tonetta
Luca Fiorini

User Flow & Implementation Analysis

Contents

Readability notes

Bold words are system-important constructs.
Underlined nouns are actors of the system.
Italic words are special keywords to denote important limits of the project.
Hyperlinks in reference to sections and subsections will be blue.
Hyperlinks to external web content will be in magenta.

Purpose

This document describes all the relevant information regarding the development of Project "TRE", which implements a rail transport management system.

The purpose of this document is to:

- Present User Flows related to the actions of performed by all users.
- Describe the (Placeholder) API interface with the payment processor.
- Accompany the documentation of the implemented processes.
- Explain and justify important implementations decisions taken.
- TODO other?

Chapter 1

TODO CLEAN CLASS DIAGRAM

Original, up-to-date files of D1 and D2 available at our Github here and Github here respectively

Chapter 2

D3

RE: Objectives

A major European rail infrastructure manager received an idea for a unified rail service scheduling and ticketing system, and has chosen us to prototype a trial run, over multiple regions in their network.

Thus, the project provides a platform that covers the needs of both passengers and train companies, as well as an underlying management system of any regional-sized rail network, which supports network managers in its administration.
It provides a set of functionalities which support the planning of train travel. The system lets passengers discover paths, view scheduling and manage tickets. On the other side, it lets train companies introduce train services and network managers administer the network.
The project coordinates the functionalities for each stakeholder into a solution that has been identified in an application accessible for the aforesaid users.

The main objectives of the project are:

Ticketing system

Subsystem which deals with tickets, from the search to the possible actions after the purchase. It includes basic operations that the passenger can perform, such as:

- Viewing tickets details.
- Purchase.
- Accessing tickets history.
- Cancellation.

Route-finding

The system shall let passengers select a starting station, a destination station, and either a departure time or arrival time.
It computes and ranks possible **routes** that best fit the user's request and permissions (for example, not permitting cargo trains to passengers).
The user must be able to compare the options based on:

- Departure and arrival times.
- Number of transfers.
- Possible transfer locations.

Virtual board

The system provides passengers access to departure and arrival boards for any station. It also shows intermediate stations and scheduled times for each individual train service.

Rail system modeling

The system provides a data-driven approach to rail network modelling, supporting an efficient administration of the infrastructure.
It shall support:

- Management of the network by the network manager.
- Management of rolling stock by the network manager.
- Submission of a new service request by train companies.
- Review of a service request by the network manager.

RE: Stakeholders

Internal

Human

Network managers:, the product owner. They manage the rail network combining the availability of train companies with passengers' needs.
They are the primary stakeholders for making functional decisions.

Non-Human

Database, where all the information used by the application are stored.

External

Human

Passengers, the main end users. They want to streamline their interactions with the train transport system, including:

- Searching for routes and schedules.
- Analyzing departure/arrival boards as well as train information.
- Purchasing tickets.

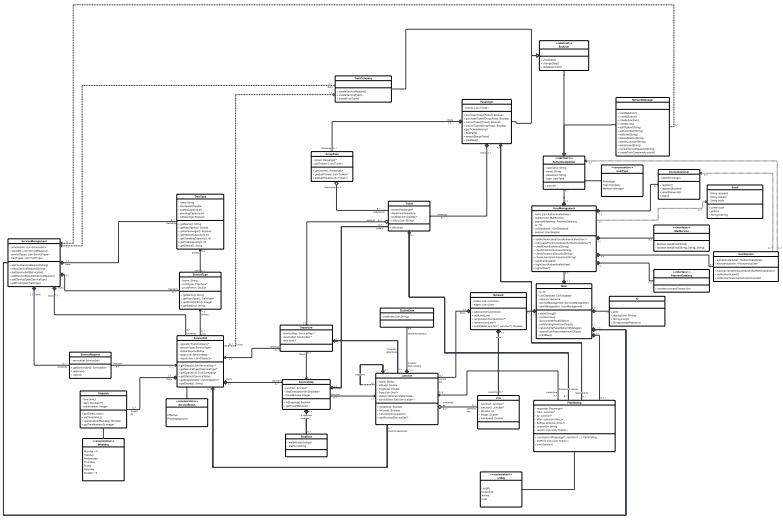
Train companies, who provide the rolling stock. They request the introduction of services. They are also expected to provide information of trains they ask to be managed.
Anonymous users, who have not yet authenticated. They are considered as passengers with restricted access to functionalities that require authentication.

Non-human

Payment gateway, which provides the underlying payment methods.

2.1 Class Diagram

In the following image, the updated class diagram, changed in order to take advatantage of the Java language and to accomodate the implementation.

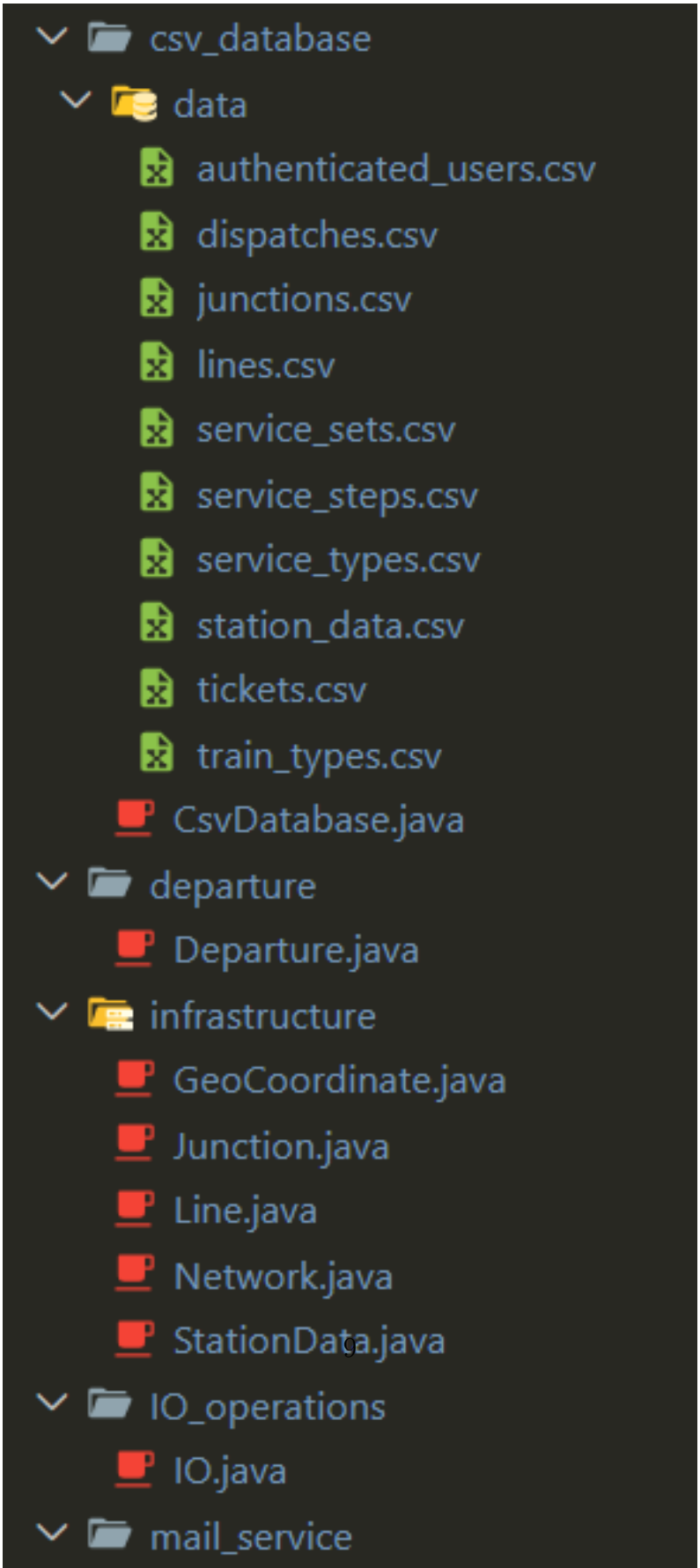


Original image can be found on our Github here

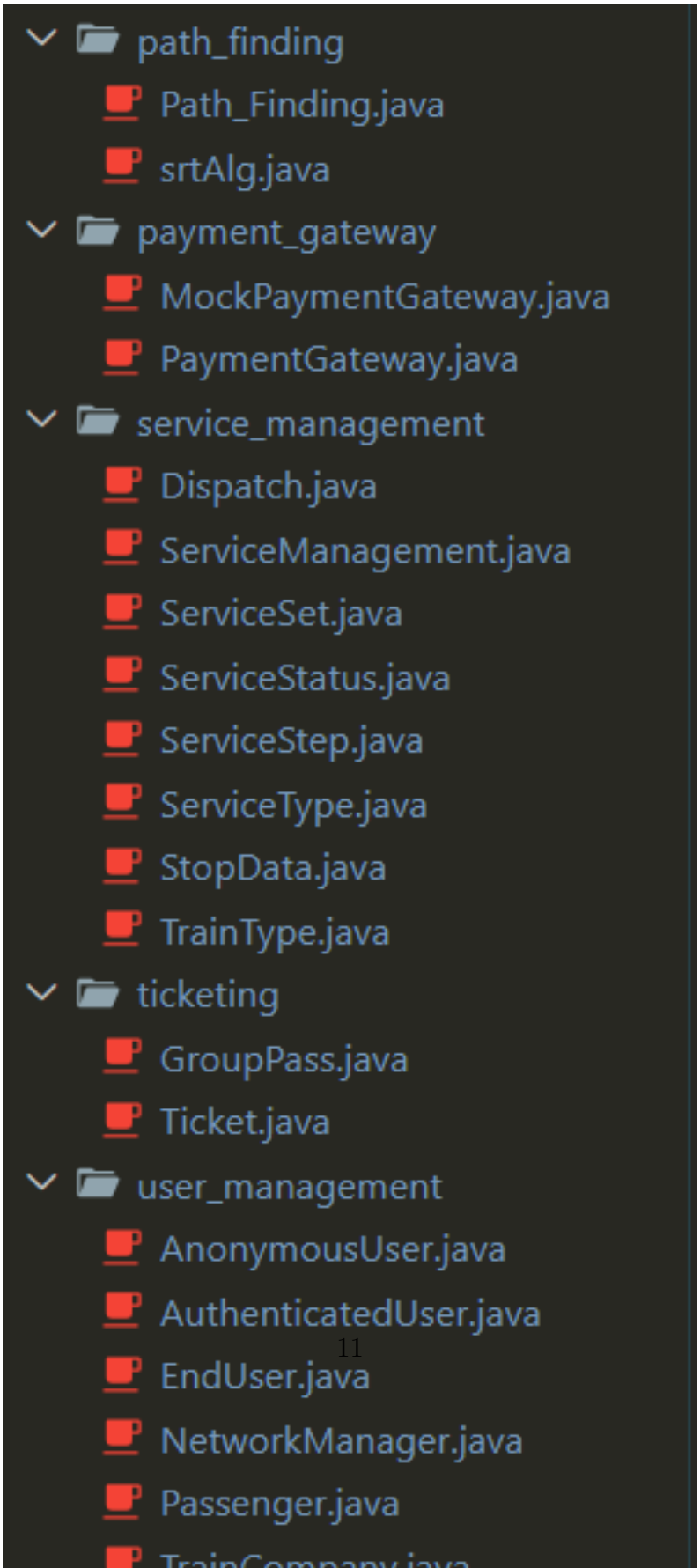
The descriptions of the classes can be found on the Documentation.

2.2 Project Structure

In the following is shown the structure of the project.


csv_database
data
authenticated_users.csv
dispatches.csv
junctions.csv
lines.csv
service_sets.csv
service_steps.csv
service_types.csv
station_data.csv
tickets.csv
train_types.csv
CsvDatabase.java
departure
Departure.java
infrastructure
GeoCoordinate.java
Junction.java
Line.java
Network.java
StationData.java
IO_operations
IO.java
mail_service

Original image can be found on our Github here


path_finding
Path_Finding.java
srtAlg.java
payment_gateway
MockPaymentGateway.java
PaymentGateway.java
service_management
Dispatch.java
ServiceManagement.java
ServiceSet.java
ServiceStatus.java
ServiceStep.java
ServiceType.java
StopData.java
TrainType.java
ticketing
GroupPass.java
Ticket.java
user_management
AnonymousUser.java
AuthenticatedUser.java
EndUser.java
NetworkManager.java
Passenger.java

Original image can be found on our Github here

TODO mettere le immagini affiancate
It is composed of:

- the main modules:
  - departure: TODO: ?;
  - infrastructure: the network and its management;
  - main: contains the runnable class, which implement the interface;
  - path_finding: the algorithm to find a path between stations;
  - service_management: the management of services;
  - ticketing: managemet of tickets information;
  - user_management: the management of users and their authentication.
- secondary tools:
  - csv_database: the actual database and its management;
  - IO_operations: input/output operation to interact with the user from terminal;
  - mail_service: integration with a possible mail service, as well as a temporary mock one;
  - payment_gateway: integration with a possible payment service, as well as a temporary mock one.

2.3 Architecture

The project is intended, in its platonic ideal, as a Client-server structure, where the client shoulders only the user interface and requests to the server, who instead is responsible for the actualy interfacing with the database and computing the responses.

Thanks to our chosen Implementation Language, the Client may be nearly any device capable of general computing, and thus we want to minimize the client-side device requirements to fully exploit our great portability.

Thus, the client is expected to perform only to the relatively light work of displaying responses and forming requests with user input to send the central server.

Not beholden to such limitations, the central server must then receive requests and perform the computatonally expensive operations, chief among them the weighted-graph-traversal over the train nework, but also loading and handling the information from the database.

Unfortunately, due to both skill and time limitations, the project is implemented as a temporary measure in a monolithic form, while still taking care to expect and enforce the necessary separation and operations for a hypothetical future division and upgrade to a client-server system ready for deployment.

2.4 Implementation Language

We chose **Java** for the operational part and **Doxygen** to generate documentation dynamically.

Our motivation for **Java** was the extreme portability and the very module-friendly structure, allowing us to experiment and tinker with different implementations of the same expected behaviour in a plug-and-play fashion. The garbage collector feature of **Java** also ensures us to spend more time working of the functioning of our systems rather than the management of their lifetimes.

2.5 Documentation

Doxygen was chosen due to its in-built support for **Java** development and for its low overhead cost during code development, as the comment-tag system requires both very little time and brain-power to make, while still being very sight-readable in the code files.
It is available at our github at the following link TODO: add link.

2.6 Project Dependencies

- JDK v25.0.3 or above

2.7 Database

To save data between different we used a database saved as several CSV files. It will be loaded at startup, containing at least the account of the network manager, with default information. The data will be managed within the code and saved to the database only at the end of the program.

The database contains one file for each set of data:

- authenticated users,
- dispatches,
- junctions,
- lines,
- service_sets,
- service_types,
- station data,
- tickets,
- train_types.

TODO: fix dimensions
The authenticated user conatins the following fields:

```
id,username,email,password,userType
0e3c90bc-0861-4019-8e82-8b56c0275106,Admin,admin@dispatchnet.it,VerySecurePassword1234,NetworkManager
```

Original image can be found on our Github here

The dispatch conatins the following fields:

```
serviceSetId,trainNumber,dispatchTime,runningDays
75a4546d-6b6b-41c3-b823-ae25be53637e,1001,05:04,WEDNESDAY;TUESDAY;FRIDAY;THURSDAY;MONDAY;SATURDAY
```

Original image can be found on our Github here

The junction contains the following fields:

```
id,name,latitude,longitude,stationDataId
bbf88b8f-97cf-4c84-b43b-198cccf5a9fb,Trento,46.072194035019535,11.118837720509594,699e0587-7686-473c-9e72-70f1de3361a1
```

Original image can be found on our Github here

The line contains the following fields:

```
id,junction1Id,junction2Id,lengthMeters,maxSpeedKpH,nTracks
84a99b0c-fc4b-4338-9af1-5568cb80b8cf,bbf88b8f-97cf-4c84-b43b-198cccf5a9fb,1ef67fd5-9fb3-4315-9125-5394ef91a75e,3100,50,1
```

Original image can be found on our Github here

The service set contains the following fields:

```
id,companyId,serviceTypeId,status,isRequest
75a4546d-6b6b-41c3-b823-ae25be53637e,beb82c42-5ee6-41dd-8888-a2744364d45a,a156105f-e0f1-41db-8386-35cfb147cb7a,Effective,false
```

Original image can be found on our Github here

The service step contains the following fields:

```
id,serviceSetId,stepIndex,junctionId,travelMinutes,platform,waitMinutes
0a1c4f16-c96b-42ce-83cb-965149581703,75a4546d-6b6b-41c3-b823-ae25be53637e,0,bbf88b8f-97cf-4c84-b43b-198cccf5a9fb,0,1 Tronco Sud,1
```

Original image can be found on our Github here

The service type contains the following fields:

```
id,commercialName,trainTypeIdentifier,centsPerKm
a156105f-e0f1-41db-8386-35cfb147cb7a,REG,Minuetto,50
```

Original image can be found on our Github here

The station data contains the following fields:

```
id,junctionId,platforms
6fdf3c23-cd1d-46ce-8363-6d9104a8dd9f,1ef67fd5-9fb3-4315-9125-5394ef91a75e,1
```

Original image can be found on our Github here

The ticket contains the following fields:

Original image can be found on our Github here

TODO add ticket row

The train type contains the following fields:

```
identifier,seatedCapacity,standingCapacity,isPassenger
Minuetto,120,50,true
```

Original image can be found on our Github here

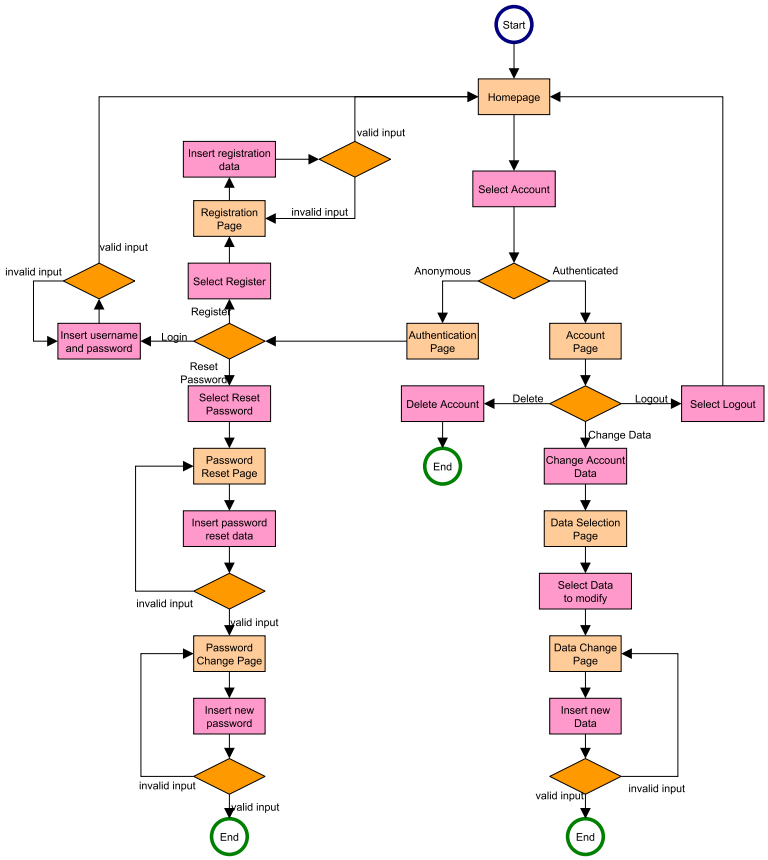
2.8 User Flows

2.8.1 User Management

The following user diagram describes the actions related to the user management and user authentication, common to all the users.

It provides different pages depending on the type of user. An authenticated user (I.E. passenger or train company, not network manager) can delete, view data (Account Page), change data or logout (also the network

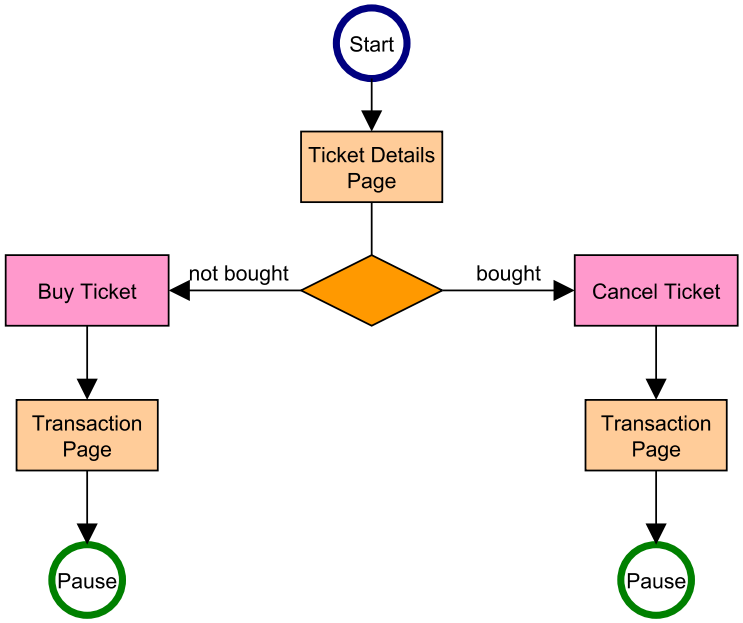
manager).
An anonymous user can register, reset its password or login.



Original image can be found on our Github here

2.8.2 Ticketing

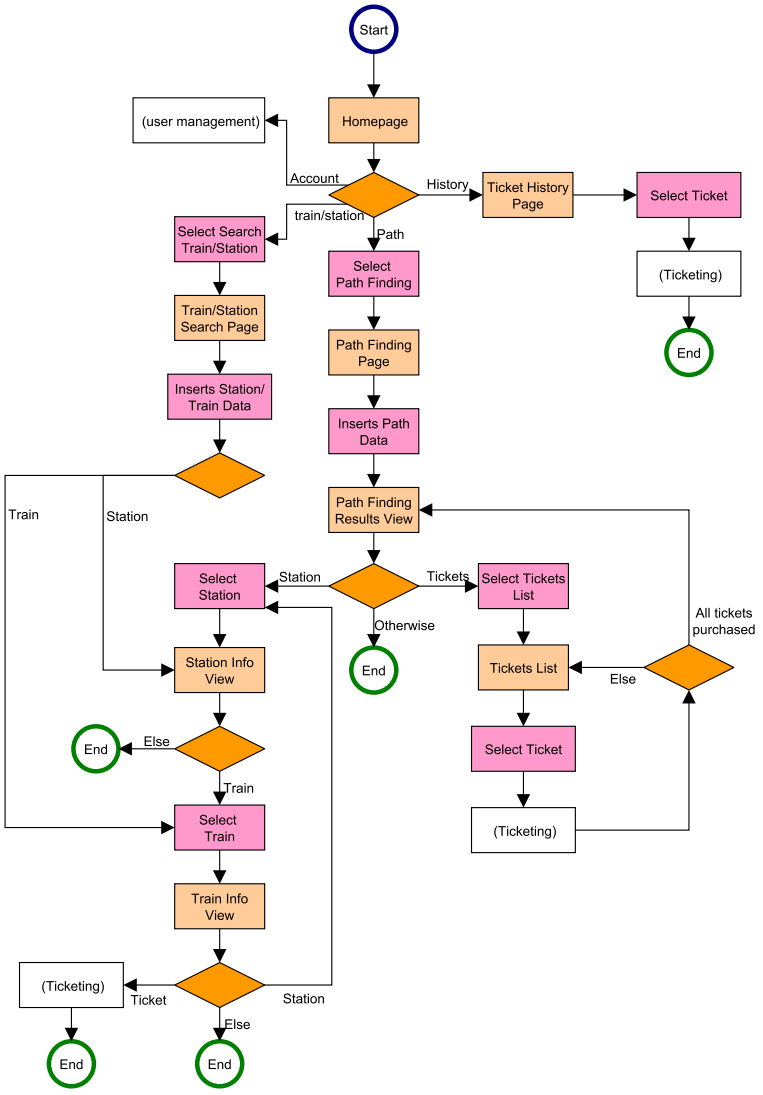
The following user diagram describes the flow of the ticketing component, started from the Passenger's diagram.



Original image can be found on our Github here

2.8.3 Passenger

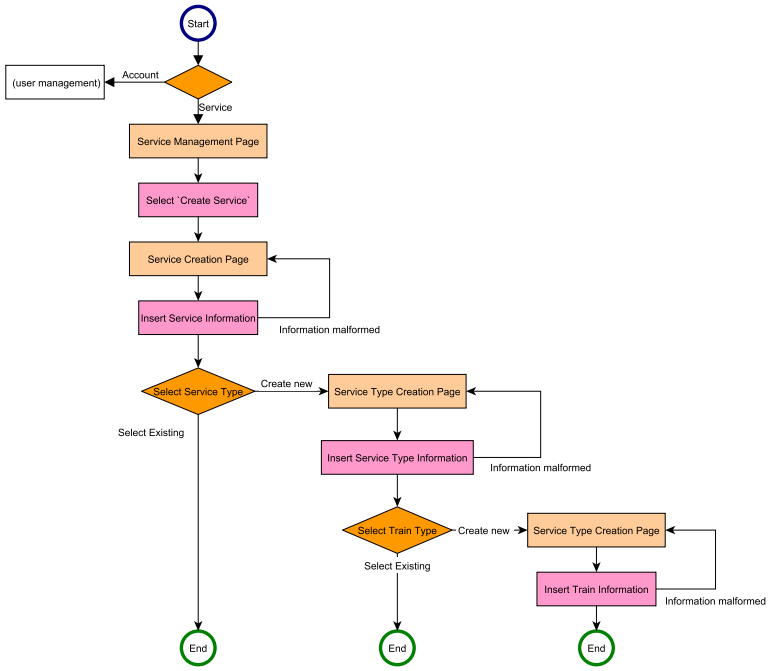
The following user diagram describes the actions the passenger can take. It can go to User Management, view its ticket history or start a path finding procedure. In the last two it can move to the Ticketing flow.



Original image can be found on our Github here

2.8.4 Train Company

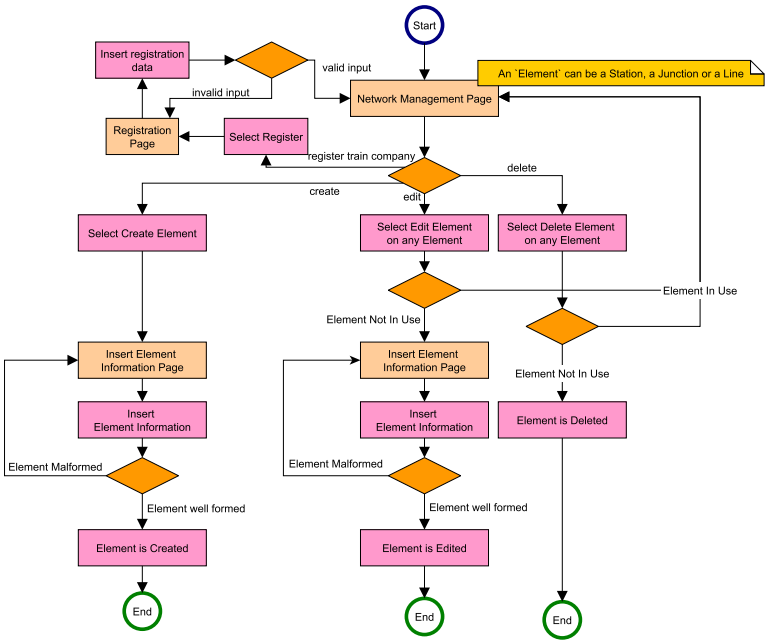
The following user diagram describes the actions the train company can take. It can go to User Management or to the service management.



Original image can be found on our Github here

2.8.5 Network Manager

The following user diagram describes the actions the network manager can take: managing the network and its elements.



Original image can be found on our Github here

2.9 APIs

As this is a mock implementation, no real API has been implemented. Instead, we opted to build fake endpoints in the form of methods, so we could test the validity of our underlying data model. All client-facing operations are found within the `UserManagement` package, in the form of class methods grouped by their relevant user. The final implementation is planned to be a REST API and will adhere to the OpenAPI standard.

External APIs have been modeled as interfaces with mock implementations. The interface paradigm allows us to use valid invocations towards a fake implementation that can easily be swapped at a later date by a real implementation. Specifically, we have implemented the following fake external APIs

- `mail_service`: The mail service
  - Interface: `MailService`
  - Mock implementation: `MockMailService`
- `payment_gateway`: The payment gateway
  - Interface: `PaymentGateway`
  - Mock implementation: `MockPaymentGateway`

The mock implementation of payment gateway also has non-deterministic behavior allowing us to test adequate handling of real-world failure conditions.

2.10 GitHub repo

TODO

2.11 Code Preview

2.12 Team roles

We divided the project into 3 natural modules, so that everyone worked on every aspect of software engineering flow. Following, specific Roles and

Activities

Table 2.1: Roles and Activities

Team member	**modules**	**Roles and Activities**
Alessio Zanon	Path finding, Info view, Ticketing	Latex expert
Leonardo Tonetta	User authentication and management, Ticketing	Team leader: monitoring deadlines, submitting deliverables, scheduling meeting
Luca Fiorini	Network and Service management	Railway expert

Table 2.2: Workload and distribution

Team member	**D1**	**D2**	**D3**
Alessio Zanon	34	33	33
Leonardo Tonetta	33	34	33
Luca Fiorini	33	33	34